

Automated OS-Level Verification for RISC-V Softcores: Bridging FPGA and Linux KernelCI

Julio N. Avelar, Marcio Godoi, Ana L. P. Costa, Rodolfo Azevedo, and Sandro Rigo

Institute of Computing - Universidade Estadual de Campinas (UNICAMP), Brazil

{j241163, m252667, a249728}@dac.unicamp.br, {rodolfo, sandro}@ic.unicamp.br

Abstract—The RISC-V ISA has driven a proliferation of open-source processor implementations with highly customizable features. Existing hardware verification methodologies validate ISA compliance and microarchitectural behavior, but often fail to expose system-level defects that emerge only under a full operating system. Conversely, software-driven continuous integration (CI) platforms such as KernelCI assume a stable hardware foundation, leaving a critical verification gap for emerging RISC-V softcores. This paper argues that, for an open ISA, hardware and software should be co-evolved within a single CI loop rather than validated in isolation, and develops this into a concrete methodology we term *hybrid HW/SW CI*. It builds on two principles: treating an FPGA-hosted soft-core as a *persistent CI runner* rather than a one-off prototype, and closing a *bidirectional co-evolution* loop in which an evolving soft-core and software stack are exercised against one another on real hardware at every commit. We realize it by bridging two previously disjoint CI systems, ProcessorCI (HW) and KernelCI (SW), through an FPGA-in-the-loop platform built on the LiteX SoC generator, and evaluate it across six diverse RISC-V implementations: Rocket, VexIIRiscv, NaxRiscv, CVA6, BlackParrot, and OpenC906. By exposing a concurrency fault in a quad-core Rocket configuration that passed single-core simulation and boot tests, we show the methodology can identify defects invisible to either domain in isolation.

Index Terms—RISC-V, Continuous Integration, Hardware Verification, FPGA Prototyping, System-on-Chip, Linux

I. INTRODUCTION

The adoption of open Instruction Set Architectures (ISAs), particularly RISC-V, has transformed processor development. Unlike proprietary ecosystems with few implementations, RISC-V hosts hundreds of open-source cores, from small embedded systems to multicore processors capable of running full operating systems such as Linux. Many cores also derive multiple architectural configurations from a single base design: cache hierarchy, core count, MMU and IOMMU support, word size, and peripherals can be tuned for specific performance, area, or power targets. This flexibility is a central advantage of the ecosystem, but it greatly expands the design space and complicates validation.

Processor verification is historically one of the costliest stages of hardware development [1]. Many methodologies address it, including compliance suites, formal verification, random instruction generation, co-simulation with instruction

set simulators, and continuous integration, primarily validating ISA correctness and microarchitectural behavior. These approaches, however, have limited visibility into defects that appear only at the system level. A full operating system exercises components rarely stressed by traditional flows, such as virtual memory, exception handling, cache hierarchies, interrupt controllers, timers, and bootloaders. Consequently, cores that pass compliance tests and simulations may still fail during OS boot or under complex workloads.

In parallel, the software community has consolidated continuous integration to validate operating systems at scale. Projects such as KernelCI [2] automate the compilation, deployment, booting, and testing of the Linux kernel across many architectures and platforms. Yet a gap separates the two validation levels: hardware-oriented CI verifies the processor in isolation, whereas OS-oriented CI assumes a stable, previously validated platform. Automated OS-level validation therefore remains underexplored for RISC-V soft-cores, especially on FPGAs, leaving an intermediate zone where subtle hardware faults remain invisible until a complete software stack runs.

This work presents an automated OS-level verification framework for RISC-V soft-cores implemented on FPGAs and uses it to develop and evaluate a broader methodology we call *hybrid HW/SW CI*. Rather than treating software tests as a fixed regression suite run against evolving hardware, the framework closes a bidirectional loop in which an evolving soft-core and an evolving software stack are exercised against one another on real hardware at every commit. The methodology enables the systematic evaluation of different RISC-V implementations and architectural configurations (cache variations, virtual memory support, and multicore setups) while automating the traditionally manual steps of platform bring-up. As detailed in the following sections, the specific contributions of this paper are: (1) A methodology for hybrid HW/SW continuous integration; (2) An open-source, end-to-end realization of this methodology that bridges hardware and software flows through an FPGA-in-the-loop platform; (3) an evaluation across six diverse RISC-V cores and multiple configurations. The remainder of this paper is organized as follows. Section II reviews related work in open-source hardware validation. Section III provides the background on the utilized toolchains. Section IV defines the conceptual framework of hybrid HW/SW CI. Section V details the methodology and the automated bridging infrastructure. Section VI describes the experimental setup. Section VII presents the evaluation results

and failure characterization. Finally, Section VIII concludes the paper.

II. RELATED WORK

Open-source hardware, driven largely by RISC-V, has reshaped how processors are designed and studied [3]. System-on-chip generators accelerate the creation of custom designs and their evaluation on FPGAs [4], [5], and the community has produced increasingly complex cores that run complete operating systems [6], [7]. As new features and custom extensions accumulate [8], [9], ensuring correct hardware–software inter-operation grows harder. To detect defects before fabrication, researchers combine formal methods [10], randomized model comparison [11], and directed tests that reach states hard to trigger manually [12].

Although these techniques continue to mature [13], most analyze instructions or circuit blocks in isolation and miss systemic defects that surface only under a running operating system. The hardware community has thus begun adopting Continuous Integration (CI) practices standard in software engineering [14]: cloud-based workflows compile RISC-V cores and run regression tests automatically [15]–[17], and some efforts unify hardware and software testing. LastLayer couples RTL simulation with software in a CI flow [18], while “*Test, Build, Deploy*” applies containerized CI/CD and specification mining to open-source RTL such as PicoRV32 [19].

These efforts confirm that unifying hardware and software validation is increasingly central; yet, two aspects remain underexplored. First, FPGA prototyping is typically a discrete verification step rather than a standing execution resource embedded in software CI. Second, the software side is treated as a fixed oracle run against evolving hardware, not an artifact that co-evolves with the core. Our work targets both gaps: it keeps the FPGA-hosted soft-core online as a *persistent CI runner* and closes a *bidirectional co-evolution* loop, validating processor modifications through the boot of an unmodified mainline Linux kernel and its native test suites, with results published to the same KernelCI dashboards used by upstream maintainers.

III. BACKGROUND

In this work, we use several tools to build a soft-core verification pipeline. The selection criteria were open-source availability and ease of integration into our framework. ProcessorCI, the RISC-V boot toolchain, and LiteX are the main selected components.

A. Processor Verification and Continuous Integration

The adoption of agile development practices within the RISC-V ecosystem has motivated the application of Continuous Integration (CI) methodologies to processor design. In previous work, we proposed ProcessorCI [15], an automated infrastructure for processor validation that integrates synthesis, RTL simulation, and the collection of physical metrics. Subsequent works have expanded this approach to encompass parametric exploration [16] and large-scale verification [17].

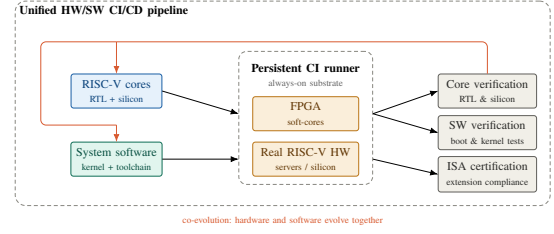


Fig. 1. Conceptual architecture of the proposed unified HW/SW CI/CD framework. A persistent CI runner — instantiated either as FPGA-hosted soft-cores or as real RISC-V hardware — exercises both hardware and software sources in the same pipeline, enabling three classes of application: core verification, software verification, and ISA extension certification. The feedback loop makes hardware and software co-evolve continuously rather than being validated in isolation.

B. Linux Boot Flow on RISC-V

The execution of a full operating system imposes significantly broader requirements than those observed in traditional processor testing. On RISC-V platforms, the Linux boot process involves multiple software layers. Following FPGA configuration, execution begins in OpenSBI [20], which is responsible for providing Machine Mode services and preparing the environment for the operating system. Subsequently, control is transferred to a bootloader, typically U-Boot [21], which loads the Linux kernel and the hardware description (Device Tree). Finally, the kernel initializes the platform subsystems, configuring virtual memory, interrupts, timers, and other resources necessary for user-space execution.

C. LiteX and Automated Platform Generation

LiteX is an open-source framework for generating FPGA-based System-on-Chip (SoC) designs. The tool provides a modular infrastructure for integrating processors, memory controllers, interconnection buses, and peripherals, enabling the construction of complete systems from parameterizable components. Through a unified interface, LiteX supports the integration of different softcores and the automatic generation of the hardware components required to run operating systems. This simplifies the creation and maintenance of FPGA-based platforms and allows for the generation of multiple hardware configurations from a single infrastructure.

IV. THE HYBRID HW/SW CI CONCEPT

The openness of RISC-V introduces a structural verification gap. In proprietary ecosystems, a small number of vendors co-design silicon and software in lockstep. In contrast, the RISC-V ecosystem typically separates these domains: hardware CI flows validate processor microarchitecture in isolation, while software CI flows assume a stable hardware target. Consequently, defects residing in the intermediate zone—such as subtle MMU or interrupt controller faults that pass compliance suites but fail under a full OS boot—often remain invisible to both isolated CI pipelines.

This work proposes that hardware and software for an open ISA should not be validated in isolation but co-evolved within a single continuous-integration loop. We refer to this

general idea as *hybrid HW/SW CI*: a methodology in which evolving hardware and an evolving software stack are exercised against one another on real hardware on every commit, so that improvements in one domain continuously drive improvements in the other. Figure 1 captures this vision. Hardware and software sources feed a *persistent CI runner*, which can be either FPGA-hosted soft-cores or real RISC-V silicon. The runner enables three classes of applications: core verification, software verification, and ISA extension certification. A feedback loop closes the system so that the two domains co-evolve rather than advance in isolation. The vision is deliberately platform- and core-agnostic. This paper materializes one concrete instance of it, scoped to RISC-V soft-cores and the system software stack.

A. Contributions Over Previous Works

While prior efforts have brought CI/CD practices to open hardware [15], [19] and have placed FPGA-based validation inside a RISC-V design flow [22]–[24], the hybrid-CI idea is distinguished by two principles that, taken together, reframe what the CI loop is for.

1) Bidirectional co-evolution of hardware and software.

Existing work overwhelmingly treats the software side of validation as a *fixed* regression suite run against evolving hardware: the software is an oracle, and only the hardware is under development. The hybrid loop instead treats both sides as first-class, co-evolving artifacts in the same pipeline, with bidirectional feedback, i.e., software test outcomes inform the hardware flow, while changes to the hardware trigger re-validation of the software that targets it. This is the feedback path drawn in Figure 1, and it is the mechanism by which the two domains mature together rather than in alternating, manually synchronized steps.

2) Real hardware as a persistent CI runner.

Prior approaches generally use hardware prototyping as a discrete, one-off verification step: a design is placed on a board, checked, and removed. The hybrid loop instead keeps the target online as a *persistent* execution resource, which is the central runner of Figure 1, that behaves like any other runner in a software CI system: continuously available to receive deployments and return results. The hardware thereby becomes a standing target in the software CI infrastructure rather than a transient artifact of a verification run.

V. METHODOLOGY

To materialize the vision of a hybrid continuous integration (CI) between hardware and software and eliminate the gap between synthesis and operating system validation without human intervention, an end-to-end automation pipeline has been developed. The main challenge of this integration lies in the fact that any architectural modification to the processor, such as the addition of a peripheral, memory map alterations, or the complete replacement of the core, requires corresponding updates to the system’s software stack.

As illustrated in Figure 2, the framework solves this problem by acting as a bridge between two existing CI systems: **Proces-**

sorCI [15] (hardware flow) and **KernelCI** [2] (software flow). This environment is integrated through an *FPGA-in-the-loop* platform, which acts as a persistent *runner* in the integration cycle, focusing specifically on the verification of RISC-V *soft-cores* and the real-time execution of the operating system. The methodology enables this flow through a strict separation between configuration and execution, combined with dynamic information transfer between hardware and software.

A. Infrastructure Abstraction and Automated Generation (ProcessorCI)

To manage a diverse matrix of platforms and *soft-cores*, the framework employs a hierarchical configuration system based on YAML files, divided into two orthogonal domains:

- **Board Configurations:** Describe the physical characteristics of the platform (clock frequency, I/O interfaces, Ethernet controllers).
- **Core Configurations:** Define the processor architecture (CPU variant, ISA extensions, cache memory sizes).

For distributed execution, the system relies on an inventory management layer. Each CI host maps the connected FPGA boards using USB serial numbers, JTAG interfaces, and MAC/IP addresses. When ProcessorCI initiates a task (triggered by a *commit*), the system allocates the appropriate board and triggers a Python script that processes the YAMLs to build the generation command for the LiteX framework. This process abstracts RTL code generation and invokes synthesis tools to produce the final *bitstream*, in addition to extracting the design’s physical metrics (area, power, and maximum frequency).

B. Dynamic Transfer Between Hardware and Software

The most critical step in ensuring co-evolution between the teams is ensuring that the *bootloader* and Linux kernel remain synchronized with the newly synthesized *bitstream*. Since the memory map varies with each build, the pipeline executes the following automated steps:

- 1) **CSR Extraction:** LiteX exports the memory map of the Control and Status Registers (CSRs) and peripherals in JSON and CSV formats.
- 2) **Device Tree Generation:** A script converts these files, reconstructing the hardware topology into a *Device Tree Source* (.dts) file.
- 3) **Firmware Compilation:** The .dts is compiled into a *Device Tree Blob* (.dtb), which is dynamically injected into the OpenSBI build, producing a custom boot firmware (fw_jump.bin).

This technique eliminates the traditional kernel *panic* failures caused by outdated static descriptions.

C. System Deployment and FPGA-in-the-loop (KernelCI)

With the *bitstream* programmed via JTAG, the board becomes operational as an active *runner*. The LiteX BIOS performs initial checks (memory and network) and automatically loads OpenSBI, the Linux Kernel, and the *rootfs* via TFTP protocol. Storage is decoupled from the hardware: the system

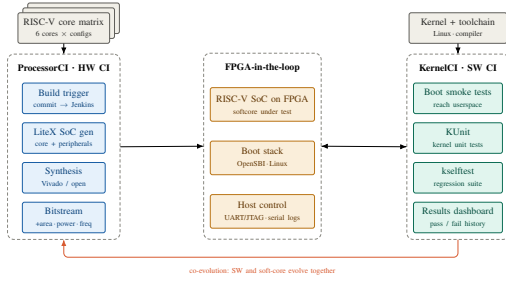


Fig. 2. Architecture of the unified HW/SW Continuous Integration framework. A configuration matrix feeds ProcessorCI, which synthesizes the LiteX SoC and programs the FPGA. The *FPGA-in-the-loop* platform boots the full stack (OpenSBI/Linux) and acts as a persistent runner, where KernelCI orchestrates tests via the serial console. The results feed the bidirectional co-evolution loop.

operates on a RAMFS and persistent data is mounted via NFS, completely eliminating manual SD card flashing.

Once OpenSBI initializes *Machine Mode* and the kernel reaches *userspace*, **KernelCI** takes control. The runner continuously monitors the serial UART interface to validate the boot *smoke test* and initiates regression suites. Internal kernel tests are validated via KUnit, while the userspace interface is attested by kselftest. Results and logs are aggregated and sent to the KernelCI dashboard, closing the bidirectional feedback loop that enables simultaneous HW and SW development.

VI. EXPERIMENTAL SETUP

We deployed and validated the infrastructure in a dedicated heterogeneous laboratory environment to evaluate the scalability, hardware independence, and viability of the proposed framework. Our setup isolates computation-heavy tasks from physical hardware management, ensuring a reliable execution loop.

A. Laboratory Infrastructure Topology

Orchestrating a continuous integration farm with real FPGA hardware requires a robust physical and network topology to prevent bottlenecks and hardware degradation. Figure 3 illustrates the physical architecture deployed in our laboratory to support the automated pipeline.

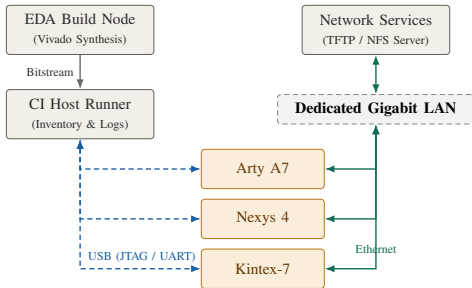


Fig. 3. Physical laboratory infrastructure. The CI Host Runner manages FPGA programming and serial logging via USB, while a dedicated Network Services node provides the OS payload and *rootfs* over Ethernet.

The infrastructure is logically and physically partitioned. Heavy computational tasks, such as generating the RTL and

executing the Xilinx Vivado synthesis flows, are dispatched to dedicated Electronic Design Automation (EDA) build nodes equipped with high memory capacity.

Once the hardware synthesis completes, the artifact (*bitstream*) is transferred to the CI Host Runner. The runner acts as the physical gateway to the board farm. It interfaces with the FPGAs exclusively via USB, utilizing multiple channels of FT2232-based bridges to perform JTAG programming and capture UART serial logs simultaneously.

Crucially, boards are physically wired to a dedicated Local Area Network (LAN) managed by a Network Services node. This node hosts a DHCP server, a TFTP server containing the OpenSBI and Linux kernel payloads, and an NFS server hosting the Linux root filesystem (*rootfs*). This physical decoupling guarantees that network boot configurations do not interfere with external laboratory networks and provides the Gigabit bandwidth necessary to quickly serve the OS payloads to multiple soft-cores booting concurrently.

B. Hardware and Software Validation Matrix

The real-time operational evaluation utilized three distinct active platforms in the matrix: Digilent Arty A7-100T, Digilent Nexys 4 DDR, and OpenSource SDR Lab Kintex-7. Additionally, higher-capacity FPGAs, such as the Xilinx VC709 and ZCU102, are already mapped in the system’s inventory for future evaluations involving larger-scale *multicore* architectures and larger area cores.

In the hardware scope, the validation matrix was configured to analyze six distinct open-source RISC-V processors: VexI-IRiscv, CVA6, NaxRiscv, Rocket, BlackParrot, and OpenC906. To explore scalability and concurrency, the Rocket processor was evaluated in single-core, dual-core, and quad-core configurations, whereas the remaining processors were instantiated as single-core systems. Logic synthesis and routing for all designs were performed using Xilinx Vivado 2024.

In the software layer, validation was executed using OpenSBI v1.3 as isolated firmware for *Machine Mode* execution and a minimal userspace generated via Buildroot 2024.02, containing exclusively the structural dependencies (*rootfs*) required to invoke the validation tools. The operating system was an unmodified mainline Linux kernel. The version string `7.1.0-rc6-gddeb000a8a` corresponds to a specific mainline development commit hash utilized at the time of testing, ensuring a modern, pre-release staging environment without requiring prior platform-specific patches. The software testing methodology encompassed a full execution of 117 KUnit test suites, comprising thousands of individual assertions—for instance, the `printf` suite alone executed 492 discrete subtests. These suites comprehensively covered core data structures (e.g., linked lists, hash tables, FIFOs), cryptographic algorithms (e.g., Blake2s, SHA1, SHA256), file system operations (e.g., ext4, FAT), and low-level kernel APIs (e.g., power management, clock frameworks, IRQ handling), alongside targeted kselftest modules for CPU hotplugging, FPU state, and system memory footprint.

Hardware performance metrics (area utilization in LUTs/Flip-Flops, power estimation, and maximum achievable frequency) were automatically extracted from the post-synthesis reports generated by Vivado through the ProcessorCI flow. Software resilience metrics (boot time and success/failure rates of unit tests) were obtained through the automated analysis of logs captured uninterruptedly by KernelCI via the UART interface across multiple interactive test suites.

VII. RESULTS AND DISCUSSION

The primary measure of success for a continuous integration infrastructure is its ability to reliably automate complex development workflows across heterogeneous hardware architectures. The proposed framework successfully generated, synthesized, and booted the complete software stack for the entire processor-FPGA matrix evaluated in this work, demonstrating the feasibility of employing an *FPGA-in-the-loop* approach for continuous operating system validation.

A. Hardware Synthesis Metrics

The framework seamlessly adapted to the resource requirements of each processor implementation without requiring manual modifications to synthesis constraints or top-level wrappers. Table I summarizes the synthesis results obtained for the OpenSource SDR Lab Kintex-7 platform. It is important to note that these metrics account for the entire System-on-Chip (SoC) footprint, including the CPU core, memory controllers, AXI/Wishbone/TileLink interconnects, and standard peripherals (UART, Ethernet MAC, etc.), as the same SoC base infrastructure was utilized for all evaluated implementations.

The abstraction provided by LiteX proved highly effective in managing the integration particularities of each processor architecture, including the interfacing of Rocket’s TileLink interconnect with the system AXI crossbar and the integration of the NaxRiscv memory subsystem. More importantly, the dynamic generation of the Device Tree completely eliminated configuration errors traditionally associated with manual platform integration. Switching the deployment target from the entry-level Arty A7-100T to the Kintex-7 platform required modifying only a single parameter in the build configuration. The pipeline automatically regenerated the memory map, rebuilt the Device Tree, and successfully booted Linux without any addressing faults arising from differences in the physical memory organization.

B. Prototyping Experience and LiteX Integration

Although LiteX significantly lowers the barrier to entry for SoC generation, integrating new hardware targets and complex out-of-order processors still presents practical engineering challenges. As a direct contribution to the open-source ecosystem, we actively developed support for the OpenSource SDR Lab Kintex-7 board and integrated it into the LiteX framework during the course of this research. Bringing up a new FPGA platform requires accurately defining physical pin assignments, clock domains, and peripheral interfaces, most notably aligning the Ethernet PHY constraints required to

enable the TFTP/NFS network boot mechanisms employed by our continuous integration infrastructure.

Nevertheless, we observed that once the initial board support had been upstreamed into the LiteX ecosystem, extending the CI pipeline to incorporate the new hardware became remarkably straightforward. The YAML-based abstraction allowed the newly integrated Kintex-7 platform to participate immediately in the validation matrix without requiring any modifications to the underlying pipeline logic.

Regarding processor integration, LiteX provides native support for a broad range of open-source processor cores. While integrating relatively simple in-order processors, such as VexRiscv, is largely a plug-and-play experience, incorporating more sophisticated architectures, such as NaxRiscv, introduces considerably more demanding architectural requirements. These integrations require appropriate interconnect wrappers, including AXI, TileLink and Wishbone interfaces, as well as carefully defined memory coherence domains and accurate interrupt controller mappings. LiteX, however, mitigates much of this complexity by providing a comprehensive library of preconfigured wrappers, substantially reducing the engineering effort required to assimilate new processor cores into the ecosystem.

Within this highly heterogeneous environment, the automated CSR extraction and dynamic Device Tree generation mechanisms proved to be fundamental for minimizing the friction inherently associated with these complex integrations. By ensuring that the software stack accurately reflects every modification introduced into the underlying hardware, this automation completely eliminates repetitive and error-prone manual synchronization tasks, thereby enabling a truly continuous hardware/software co-development workflow.

Despite the comprehensive support provided by LiteX, the quality and stability of upstream processor integrations can vary significantly. During our evaluation, we encountered overwhelming integration failures with the BlackParrot and OpenC906 implementations currently available in the LiteX ecosystem. Both cores exhibited critical synthesis errors or immediate boot-stage exceptions that prevented the generation of a viable Linux-capable bitstream. Consequently, these two architectures could not advance to the OS-level validation phase. These failures highlight a critical reality of the open-source hardware landscape: upstream availability and isolated RTL simulation do not guaranty system-level operational stability, further reinforcing the necessity for continuous, automated integration pipelines.

C. Operating System Validation

Beyond identifying microarchitectural defects in mature processor implementations, the primary contribution of this work is the establishment of an automated methodology realized through an infrastructure for the continuous validation of hardware under active development. By autonomously deploying the software stack through TFTP and NFS, the proposed infrastructure sustained long-running software validation workloads while completely eliminating the bottlenecks

TABLE I
COMPREHENSIVE AUTOMATED VALIDATION MATRIX (TARGET: OPENSOURCE SDR LAB KINTEX-7)

SoC Core	LUTs / FF	Linux Boot	KUnit	kselftest	Total Time	Failure Characterization
VexIIRiscv (1x)	28418 / 24705	Pass	Pass	Pass	~42m	Stable baseline execution.
Rocket (1x)	34760 / 19246	Pass	Pass	Pass	~45m	Stable baseline execution.
Rocket (4x)	104280 / 57738	Pass	Fail	N/A	N/A	Deterministic lockless I/O concurrent fault.
CVA6 (1x)	53756 / 34179	Pass	Pass	Pass	~50m	Stable baseline execution.
NaxRiscv (1x)	33436 / 24035	Pass	Pass	Pass	~48m	Stable baseline execution.
BlackParrot	N/A	Fail	N/A	N/A	N/A	Synthesis mapping/integration error in LiteX.
OpenC906	N/A	Fail	N/A	N/A	N/A	Immediate exception during OpenSBI handoff.

Note: Hardware metrics reflect the entire SoC footprint. Total time includes synthesis, network boot, and regression testing.

and operational overhead associated with physical storage media, such as manually programming SD cards.

Throughout the continuous evaluation process, the framework successfully executed complete Linux boot sequences and regression test suites using the *mainline* Linux kernel version 7.1.0-rc6. Integrating the HW and SW CI workflows fundamentally changed the validation timeline. Running a comprehensive OS-level validation, such as the full execution of the 117 KUnit test suite, is highly computationally intensive. On an entry-level platform like the Arty A7 equipped with a single-core Rocket processor, this full software validation loop takes approximately 45 minutes to complete.

This extended execution time rivals and frequently exceeds the 5 to 25 minutes required for the Vivado RTL synthesis of the underlying SoC. Rather than a temporal asymmetry favoring fast software iteration, this reality highlights the critical value of the *Persistent CI Runner* concept. Because the hardware and software validation pipelines are completely decoupled, a successfully synthesized FPGA bitstream is cached, and the board remains continuously online. Consequently, software developers can deploy an updated Linux kernel via TFTP and initiate long-running validation suites without incurring the temporal penalty of hardware synthesis. To demonstrate this capability, the pipeline automatically intercepts the serial output of the target board, parsing the execution of unit tests directly instantiated in the soft-core memory.

```
[ 361.271387] # Totals: pass:12 fail:0 skip:1 total:13
[ 361.276842] ok 37 blake2s
[ 361.284415] KTAP version 1
[ 361.287426] # Subtest: sha1
[ 361.290255] 1..11
[ 361.334112] ok 1 test_hash_test_vectors
...
[ 1598.950354] kselftest: Running tests in cpu-hotplug
# timeout set to 45
# selftests: cpu-hotplug: cpu-on-off-test.sh
not ok 1 selftests: cpu-hotplug: cpu-on-off-test.sh # exit=127
...
[ 1599.932279] kselftest: Running tests in size
# ok 1 get runtime memory use
# 1..1
ok 4 selftests: size: get_size
```

Fig. 4. Excerpt of an automated serial log capture during a test cycle, demonstrating the OS-level validation of cryptographic subsystems and system size directly on the FPGA soft-core.

An illustrative example of a successful KUnit and kselftest verification phase, automatically parsed by the

infrastructure, is shown in Figure 4. Executing cryptographic subsystem validation (e.g., Blake2s and SHA1) through KUnit, alongside userspace memory footprint evaluations via kselftest directly on the FPGA hardware, provides a level of execution fidelity that conventional RTL simulators are unable to achieve within practical development times. These outputs are subsequently JSON-formatted and pushed to the standard KernelCI dashboard, allowing hardware engineers to track regressions using the exact same web interfaces utilized by Linux kernel maintainers.

The automated pipeline’s ability to expose complex, system-level concurrency bugs is demonstrated by a targeted case study involving the Rocket core. While single-core and dual-core configurations successfully passed all validation suites, the quad-core configuration revealed a critical and deterministic failure. Specifically, the quad-core system consistently crashed during the `printk-ringbuffer` KUnit test, which heavily stresses concurrent memory access and lockless I/O buffer management across multiple threads. To isolate the root cause, we verified that the FPGA bitstream met all strict timing constraints, ruling out physical timing violations. Furthermore, the LiteX AXI/TileLink interconnect wrappers and interrupt mappings remained identical to the stable dual-core baseline.

Because the failure is highly reproducible exclusively under heavy concurrent load and isolated from peripheral routing, the data strongly points to an architectural edge case in Rocket’s cache coherence protocol or memory consistency model rather than a platform integration artifact. This defect remains entirely undetectable in single-core RTL simulations and standard boot tests, proving the necessity of persistent, multicore OS-level validation.

D. Discussion and Architectural Comparison

The proposed automated pipeline significantly reduced the overall verification cycle by eliminating manual intervention during platform bring-up. In a conventional manual workflow, compiling the software stack, generating a consistent Device Tree, programming the FPGA, and initiating network boot mechanisms typically requires 45 to 90 minutes of dedicated engineering time per iteration, a process highly susceptible to human error. This HW/SW Hybrid CI framework reduces this entire workflow to a fully automated trigger executed in milliseconds, with the subsequent test cycle proceeding deterministically without human intervention. Moreover, since

the underlying ProcessorCI ecosystem already supports the synthesis and RTL simulation of over 100 distinct RISC-V cores, this work lays the groundwork for a massive scale-up in automated OS-level validation.

Furthermore, subjecting these processors to the stringent requirements of the KernelCI ecosystem highlights the growing maturity of the open-source RISC-V ecosystem. The ability of architectures ranging from simple 32-bit in-order implementations to complex 64-bit out-of-order processors to boot an unmodified *mainline* Linux kernel using a hardware-agnostic continuous integration pipeline demonstrates that the RISC-V ecosystem is steadily narrowing the gap with established architectures such as ARM and x86. In these mature ecosystems, hardware/software co-validation remains largely confined to proprietary infrastructures. By bridging ProcessorCI and KernelCI using entirely open-source tools, the proposed framework simplifies system-level validation and provides a practical blueprint through which emerging processor architectures can rapidly achieve ecosystem maturity via continuous, automated operating system-level validation.

VIII. CONCLUSION

This paper presents an automated OS-based validation framework that bridges the gap between hardware continuous integration and the Linux KernelCI infrastructure. By leveraging LiteX to abstract FPGA platform generation and implementing a dynamic hardware-to-software handoff mechanism based on automated Device Tree generation, the proposed methodology enables the bidirectional co-evolution of RISC-V soft processors and the system software stack.

The experimental evaluation conducted across three FPGA platforms and six processor implementations demonstrated the viability of using real hardware as a *Persistent CI Runner*. This approach substantially reduces the engineering effort required for platform bring-up and ensures that emerging open-source processor architectures can execute and co-evolve with modern operating systems in a reliable and stable manner.

Future work will aim to expand the number of evaluated processors, capitalizing on ProcessorCI's existing support for over 100 RISC-V core variations. This massive expansion of the execution matrix will be paired with larger FPGA platforms, such as the Xilinx VC709, enabling the automated validation of complex multicore topologies and the integration of formal RISC-V architectural compliance test suites directly into the *hardware-in-the-loop* validation pipeline.

REFERENCES

- [1] W. Rülling, *Circuit Verification*, pp. 210–218. Boston, MA: Springer US, 2003.
- [2] KernelCI Foundation, “KernelCI: Standardized hardware testing for the Linux kernel.” <https://kernelci.org>, 2024. Open-source project under the Linux Foundation.
- [3] J. L. Hennessy and D. A. Patterson, “A new golden age for computer architecture,” *Commun. ACM*, vol. 62, p. 48–60, jan 2019.
- [4] C. Papon, “Vexriscv: A risc-v cpu implementation in spinalhdl.” <https://github.com/SpinalHDL/VexRiscv>, 2026. Accessed: 2026-07-06.
- [5] F. Kermarrec, S. Bourdauducq, J.-C. Le Lann, and H. Badier, “Litex: an open-source soc builder and library based on migen python dsl,” *arXiv preprint arXiv:2005.02506*, 2020.
- [6] J. Zhao, B. Korpan, A. Gonzalez, and K. Asanovic, “Sonicboom: The 3rd generation berkeley out-of-order machine,” in *Fourth Workshop on Computer Architecture Research with RISC-V*, vol. 5, pp. 1–7, International Symposium on Computer Architecture Valencia, 2020.
- [7] J. Balkind, K. Lim, F. Gao, J. Tu, D. Wentzlaff, M. Schaffner, F. Zaruba, and L. Benini, “Openpiton+ ariane: The first open-source, smp linux-booting risc-v system scaling from one to many cores,” in *Workshop on Computer Architecture Research with RISC-V (CARRV)*, pp. 1–6, 2019.
- [8] M. Gazziro, J. M. d. A. Junior, O. H. A. Junior, M. R. Cavallari, and J. P. Carmo, “Design and Evaluation of Open-Source Soft-Core Processors,” *Electronics*, vol. 13, no. 4, 2024.
- [9] A. Waterman, Y. Lee, and K. Asanović, “The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 20211203,” tech. rep., RISC-V International, December 2021.
- [10] A. Armstrong, T. Bauereiss, B. Campbell, A. Reid, K. E. Gray, R. M. Norton, P. Mundkur, M. Wassell, J. French, C. Pulte, S. Flur, I. Stark, N. Krishnaswami, and P. Sewell, “ISA semantics for ARMv8-a, RISC-v, and CHERI-MIPS,” *Proc. ACM Program. Lang.*, vol. 3, jan 2019.
- [11] A. Joannou, P. Rugg, J. Woodruff, F. A. Fuchs, M. Van der Maas, M. Naylor, M. Roe, R. N. Watson, P. G. Neumann, and S. W. Moore, “Randomized testing of risc-v cpus using direct instruction injection,” 2024.
- [12] A. Jayasena and P. Mishra, “Directed test generation for hardware validation: A survey,” *ACM Computing Surveys*, vol. 56, no. 5, pp. 1–36, 2024.
- [13] V. Herdt, D. Große, E. Jentzsch, and R. Drechsler, “Efficient cross-level testing for processor verification: A risc-v case-study,” in *2020 Forum for Specification and Design Languages (FDL)*, pp. 1–7, IEEE, 2020.
- [14] M. Shahin, M. A. Babar, and L. Zhu, “Continuous integration, delivery and deployment: a systematic review on approaches, tools, challenges and practices,” *IEEE access*, vol. 5, pp. 3909–3943, 2017.
- [15] Avelar, Julio N. and Lago, Victor P. and Malaguti, Ângelo R. P. and Azevedo, Rodolfo, “ProcessorCI: Continuous integration environment for open-source RISC-V processors using FPGAs.” <https://processorci.lsc.ic.unicamp.br>, 2024. Laboratory of Computer Systems, Institute of Computing, University of Campinas.
- [16] G. Oliveira, J. N. Avelar, E. P. Bertoloti, G. Gomes, and R. Azevedo, “Processor ci inspector: Detectando parâmetros automaticamente em processadores risc-v,” in *Simpósio em Sistemas Computacionais de Alto Desempenho (SSCAD)*, pp. 97–104, SBC, 2025.
- [17] G. Gomes, J. Avelar, G. Oliveira, E. Bertoloti, and R. Azevedo, “Large-scale risc-v processor verification using automated design inspection and a generic simulation method,” in *Simpósio em Sistemas Computacionais de Alto Desempenho (SSCAD)*, pp. 181–192, SBC, 2025.
- [18] L. Vega, J. Roesch, J. McMahan, and L. Ceze, “Lastlayer: Toward hardware and software continuous integration,” *IEEE Micro*, vol. 40, no. 4, pp. 103–111, 2020.
- [19] C. Deutschbein and A. Stassinopoulos, ““test, build, deploy” - a ci/cd framework for open-source hardware designs,” in *2025 5th International Conference on Electrical, Computer and Energy Technologies (ICE-CET)*, pp. 1–6, 2025.
- [20] A. Patel and O. Contributors, “Opensbi: Risc-v open source supervisor binary interface.” <https://github.com/riscv-software-src/opensbi>, 2024.
- [21] U.-B. Contributors, “Das u-boot: The universal boot loader.” <https://www.denx.de/wiki/U-Boot>, 2024.
- [22] S. Karandikar, H. Mao, D. Kim, D. Biancolin, A. Amid, D. Lee, N. Pemberton, E. Amaro, C. Schmidt, A. Chopra, Q. Huang, K. Kovacs, B. Nikolic, R. Katz, J. Bachrach, and K. Asanović, “FireSim: FPGA-accelerated cycle-exact scale-out system simulation in the public cloud,” in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, pp. 29–42, IEEE, 2018.
- [23] K. Qin and M. Schulz, “FERIVer: An FPGA-assisted emulated framework for RTL verification of RISC-V processors,” in *Proceedings of the 22nd ACM International Conference on Computing Frontiers (CF '25)*, ACM, 2025.
- [24] S. Ahmed, A. Kropotov, R. Genovese, B. Homs, E. Merino, F. Urbani, H. Yano, I. Díaz, J. Fernandez, M. Toselli, M. Imran, M. Iqbal, N. Yaseen, Q. Abid, S. Cheema, S. Sanchez, D. Garcia, J. Cabré, M. Elyasi, and B. Salami, “Verification and validation (V&V)-in-the-loop for RISC-V design: The holistic vision of BZL.” *arXiv preprint arXiv:2604.27013*, 2026.